

Module 5: Code Optimization & Code Generation

Basic Blocks, CFG, DAG, Loop Optimizations, Data-Flow Equations & Register Coloring

Module V: Code Optimization & Target Code Generation – Complete Syllabus Topics

1. Principal Sources of Optimization (Machine-Independent)
2. Basic Blocks Partitioning Algorithm & Control Flow Graphs (CFG)
3. Directed Acyclic Graphs (DAG) Representation for Basic Blocks
4. Local vs. Global Code Optimization Techniques
5. Loop Optimizations (Loop Invariant Code Motion, Strength Reduction, Unrolling)
6. Dominators, Natural Loops & Reducible Flow Graphs
7. Data-Flow Analysis Equations (Reaching Definitions & Available Expressions)
8. Live Variable Analysis & Dead Code Elimination
9. Target Code Generation Issues & Register Allocation by Graph Coloring
10. Peephole Optimization Techniques (Redundant Load/Store, Algebraic Identities)

Topic 1 & 4: Principal Sources of Code Optimization

Code optimization transforms intermediate or target code to run faster, consume less memory, or use less power, while strictly preserving program semantics.

Optimization Technique	Mechanisms & Transformations	Before → After Example
1. Constant Folding	Evaluating operations on compile-time constants upfront.	<code>`x = 3 + 4 * 2` → <code>`x = 11`</code></code>
2. Constant Propagation	Replacing variables holding known constant values with the constant.	<code>`pi = 3.14; r = 2; area = pi * r * r` → <code>`area = 3.14 * 2 * 2`</code></code>
3. Common Subexpression Elimination	Reusing previously computed values instead of re-evaluating identical expressions.	<code>`t1 = a + b; ...; t2 = a + b` → <code>`t2 = t1`</code></code>
4. Dead Code Elimination	Removing instructions whose results are never read by subsequent code or unreachable blocks.	<code>`if (0) { do_something(); }` → <code>`/* Removed */`</code></code>
5. Strength Reduction	Replacing expensive operations (e.g., multiplication) with cheaper equivalents (addition, bit-shift).	<code>`x = y * 8` → <code>`x = y << 3`</code></code>
6. Loop Invariant Code Motion	Hoisting computations that evaluate to the same value in every loop iteration outside the loop header.	<code>`while (i < n) { x = a + b; i++; }` → <code>`x = a + b; while (i < n) { i++; }`</code></code>

Topic 2: Basic Blocks & Control Flow Graphs (CFG)

Algorithm: Partitioning Three-Address Code into Basic Blocks

Step 1: Identify the Leaders (first instructions of basic blocks):

1. The first Three-Address instruction of the intermediate code is a Leader.
2. Any instruction that is the target of a conditional or unconditional jump (`goto L`) is a Leader.
3. Any instruction that immediately follows a conditional or unconditional jump is a Leader.

Step 2: Construct Basic Blocks:

For each Leader, its Basic Block consists of the Leader and all subsequent instructions up to, but not including, the next Leader or the end of the program.

Step-by-Step Solved Problem: Basic Block Partitioning on Dot Product Code

Three-Address Code:

```
(1) prod = 0           ← Leader (Rule 1: First instruction)
(2) i = 1
(3) t1 = 4 * i         ← Leader (Rule 2: Target of goto (3) in line 9)
(4) t2 = a[t1]
(5) t3 = 4 * i
(6) t4 = b[t3]
(7) t5 = t2 * t4
(8) prod = prod + t5
(9) i = i + 1
(10) if i ≤ 20 goto (3)
(11) print(prod)      ← Leader (Rule 3: Follows conditional jump at line 10)
```

Resulting Basic Blocks:

- **Block B_1** : Instructions (1) to (2).
- **Block B_2 (Loop Body)**: Instructions (3) to (10).
- **Block B_3** : Instruction (11).

Topic 3: Directed Acyclic Graphs (DAG) for Basic Blocks

A **DAG (Directed Acyclic Graph)** is an efficient representation for basic blocks that makes local common subexpression elimination, dead code elimination, and array dependency tracking straightforward:

Leaves correspond to initial values of variables or constants.

Interior nodes correspond to operators.

Nodes have labels attached showing current identifier names having that computed value.

Topic 7 & 8: Data-Flow Analysis Frameworks

Framework	Data-Flow Transfer Equations	Direction & Confluence Operator
1. Reaching Definitions	$\text{OUT}[B] = \text{GEN}[B] \cup (\text{IN}[B] \setminus \text{KILL}[B])$ $\text{IN}[B] = \bigcup_{P \in \text{Pred}(B)} \text{OUT}[P]$	Forward Data-Flow; Union ($\cup$) confluence.
2. Available Expressions	$\text{OUT}[B] = e_ \text{GEN}[B] \cup (\text{IN}[B] \setminus e_ \text{KILL}[B])$ $\text{IN}[B] = \bigcap_{P \in \text{Pred}(B)} \text{OUT}[P]$	Forward Data-Flow; Intersection ($\cap$) confluence.
3. Live Variable Analysis	$\text{IN}[B] = \text{USE}[B] \cup (\text{OUT}[B] \setminus \text{DEF}[B])$ $\text{OUT}[B] = \bigcup_{S \in \text{Succ}(B)} \text{IN}[S]$	Backward Data-Flow; Union ($\cup$) confluence.

Topic 9 & 10: Target Code Generation & Peephole Optimization

1. Register Allocation by Graph Coloring (Chaitin's Algorithm):

Construct the **Interference Graph** where nodes represent variables/temporaries and edges connect variables that are simultaneously live at the same program point.

Find a K -coloring of the graph, where K is the number of available physical hardware CPU registers. If successful, no two interfering variables share the same register.

If the graph cannot be K -colored, select candidate variables to **spill** into RAM stack slots.

2. Peephole Optimization:

A local optimization method that examines a short slide window of target assembly instructions ("peephole") and applies immediate algebraic and redundant instruction eliminations:

Eliminating Redundant Loads and Stores:

```
MOV R0, a
MOV a, R0      ; Redundant - Eliminated!
```

Eliminating Unreachable Code: Instructions immediately following an unconditional jump without a label are deleted.

Algebraic Simplifications: Replacing $x + 0 \rightarrow x, x * 1 \rightarrow x, x * 0 \rightarrow 0$.



Top BIT Mesra Exam Questions & Answers (Module V)

Q1. State the algorithm to identify Basic Blocks and construct a Control Flow Graph. (8 Marks)

1. **Identify Leaders:** (i) First instruction of program, (ii) Target of any jump (`goto`), (iii) Instruction immediately following any jump.
2. **Construct Blocks:** For each leader, its basic block consists of all instructions up to the next leader or program end.
3. **Build Control Flow Graph (CFG):** Nodes are the basic blocks. Directed edges $B_i \rightarrow B_j$ exist if control can transfer from B_i to B_j (via conditional/unconditional jump or sequential fall-through).

Q2. Explain Loop Invariant Code Motion and Strength Reduction with code examples. (8 Marks)

- **Loop Invariant Code Motion:** Moves computations whose operand values do not change across loop iterations outside the loop header.

Example: `for (int i=0; i < 100; i++) { t = i \* 4; }` - **Strength Reduction:** Replaces computationally expensive operations (e.g., multiplication inside loops driven by induction variables) with cheaper operations (addition).

Example: `for (int i=0; i < 100; i++) { t = i \* 4; }` → `t = 0; for (int i=0; i < 100; i++) { ... t += 4; }`